

Poppin Music - Final Build Master Spec

Repo-level coding instructions with clear phases, Session Agent integration, PMR, Musical Engine, Reaper Bridge and School handoff. Designed to be used by Cursor/Grok as the "code next phase" reference.

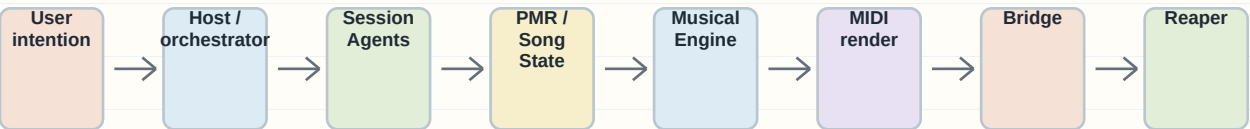
Core product sentence	Poppin Music is a local-first musical intelligence layer in front of a DAW. The user provides taste and intention; Session Agents provide musical craft; PMR preserves musical meaning; the Musical Engine renders and analyses; the DAW Bridge makes it audible in Reaper.
-----------------------	---

1. Founding principles

#	Principle
1	The interface is taste, intention, listening and selection - not music theory homework.
2	MIDI is a rendered/playable form, not the source of truth.
3	PMR is the internal language; Song State is the current song in that language.
4	Session Agents are musicians, not hidden app functions.
5	Musical Engine is deterministic tools plus music-native models.
6	DAW Bridge translates to/from Reaper.
7	Human audition/commit is the gate.
8	Song Memory feeds creativity; Identity Memory observes by default.
9	Build musicians first; the beautiful canvas is earned later.

2. Architecture in one view

Main runtime flow



The coding rule: keep these layers separate. Session Agents output PMR candidate material, not Reaper commands. Reaper-specific details live only in the Bridge.

3. Phase plan

Phase	Goal	Key deliverables	Acceptance
0 - Repo foundation	Create a clean codebase that can grow.	Repo layout, PMRSketch schema, MIDI validation/render test, local config.	Sample PMRSketch renders to valid MIDI events; no Reaper objects leak into agent outputs.
1 - Absolute V0	Hear one useful sketch in Reaper.	Prompt + tempo/key/meter/bars; one intelligence call; Harmony/Bass/Drums; PMRSketch; MIDI render; simple bridge.	Enter an 8-bar intention, generate sketch, Reaper plays three tracks, logs show PMR/MIDI/bridge.
1.1 - Session contract split	Prevent the first call from becoming a blob.	SessionAgentManifest, input/output schemas, separate Harmony/Bass/Drums modules.	Host can call Bass alone with existing harmony; all outputs validate.
1.2 - A/B/C audition + commit	Hear proposals before they become reality.	Candidate branches, audition, commit, regenerate/reject, basic undo.	Audition Harmony A/B/C, commit B; Bass hears committed harmony; undo works.
1.3 - Strong Reaper Bridge	Make Reaper the bench, not an import chore.	Named-track template, ReaScript import/replace/play, logical bridge API.	Candidate to audition track; commit to durable MIDI in correct range.
2 - Real Song State	PMR becomes canonical and versioned.	Song/Section/Track/Harmony/Performance/Decision/DAWMapping types; locks; provenance.	Keep drums; redo bass bars 5-8 as state operation.
3 - Reverse sync	Human edits in Reaper become PMR truth.	Read tracks/items/ranges; compare PMR render; normalise changed MIDI.	Move bass notes in Reaper; next Bass Agent sees edited version.
4 - Minimal UI	Pleasant working interface, not final canvas.	Prompt bar, context chips, agent rail, PMR inspector, A/B/C cards, Reaper status.	Phase 1.2 loop works without terminal.
5 - Session Agent Library	Trained agents can be installed.	Agent manifest convention, package folder, scanning, enable/disable.	Drop in agent folder; host validates and calls it.
6 - School handoff	Poppin consumes School outputs.	Export evaluation records; import promoted agents.	School exports agent package; Poppin installs/updates it.
7 - Expansion	Producer, DAW awareness, Logic/ACE.	Producer/DAW agents, plugin awareness, adapters, references, sonic analysis.	Added only after Reaper loop and agent contract are stable.

4. Suggested repo structure

```
poppin-music/
docs/
POPPIN_MUSIC_BUILD_MASTER_SPEC.md
SESSION_AGENT_SCHOOL_PRODUCT_SPEC.md
apps/host/
packages/pmr/
packages/musical-engine/
packages/session-agents/
packages/reaper-bridge/
packages/shared/
reaper/scripts/
reaper/templates/
tests/
```

5. PMR rules

Definition

PMR stands for Poppin Musical Representation. PMR is the language. Song State is the current composition written in that language. MIDI is the playable/rendered form.

Minimum Phase 1 PMRSketch

```
type PMRSketch = {
meta: { title?: string; tempo: number; meter: string; key?: string; bars: number };
intention: string;
sections: Array<{ id: string; name: string; startBar: number; endBar: number }>;
harmony: Array<{ bar: number; chord: string; role?: string; tension?: string; notes?: string[] }>;
bass: Array<{ bar: number; events: NoteEvent[]; role?: string }>;
drums: Array<{ bar: number; events: DrumEvent[]; role?: string }>;
decisions?: DecisionRecord[];
};
```

6. Musical Engine workflow

Forward



Reverse



7. Session Agent Library and School integration

Runtime boundary

Poppin consumes trained Session Agents. It does not need Session Agent School running during normal composition. Poppin can export evaluation records and import promoted agent packages.

Installable agent package format

```
agents/harmony-v1/  
manifest.json  
skills/  
prompts/  
evaluations/  
adapter.ts  
README.md
```

8. Cursor/Grok coding-agent instructions

Rule	Instruction
1	Do not expand scope unless explicitly instructed.
2	Implement the next smallest phase slice.
3	Keep PMR separate from MIDI and Reaper.
4	Keep Session Agents separate from the host app.
5	Keep Reaper commands inside the Bridge only.
6	Add tests for schemas, rendering and bridge outputs.
7	Prefer boring reliable code over clever architecture theatre.
8	Update this document when a decision changes.
9	Produce run command and manual test recipe.
10	Report done/mocked/not implemented.

9. North-star manual test

Open Reaper template. Start Poppin local app. Enter: "8 bars, quiet and reflective, slightly unresolved, simple enough for a melody." Generate harmony, bass and drums. Send to Reaper. Hear it. Regenerate if weak. Commit if good.

The pass condition is not merely valid JSON. The pass condition is: "Oof. Keep that."